

Jenkins Master + Agent + Docker CI/CD

Step-by-Step Replication Manual

Setup covered: Jenkins Master EC2 + Jenkins Agent EC2 + SSH connection + Docker build + Docker Compose deployment.

1. Architecture



2. Create Jenkins Master EC2

1. Create an EC2 instance for the Jenkins master.
 - Use a Linux/Ubuntu image.
 - Allow SSH (port 22).
 - Allow Jenkins (port 8080).
 - For testing, port 8080 can be opened to 0.0.0.0/0.
2. SSH into the master:

```
ssh -i your-key.pem ubuntu@MASTER_PUBLIC_IP
```

3. Install Java on the Master

3. Install the Java JDK using the official Jenkins Linux installation instructions.
4. Verify Java:

```
java -version
```

4. Install Jenkins

5. Install Jenkins using the official Jenkins Linux installation instructions.
6. Check Jenkins:

```
sudo systemctl status jenkins
```

- The service should show: Active: active (running)
7. Enable Jenkins at boot:

```
sudo systemctl enable jenkins
```

5. Open Jenkins

8. Open Jenkins in a browser:

```
http://MASTER_PUBLIC_IP:8080
```

9. On the Jenkins master, get the initial administrator password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

10. Copy the password into the Jenkins unlock page.
11. Select Install suggested plugins and wait for installation to finish.
12. Create the Jenkins administrator account and log in.

6. Create Jenkins Agent EC2

13. Create a second Linux EC2 instance.
14. Example name:

```
agent-jenkins
```

15. SSH into the agent:

```
ssh -i your-key.pem ubuntu@AGENT_PUBLIC_IP
```

7. Install Java on the Agent

16. Install the Java JDK using the official Jenkins/Linux instructions.
17. Verify Java:

```
java -version
```

Jenkins itself is not installed on the agent. The agent only needs Java and the tools required by the pipeline.

8. Create an SSH Key on the Master

18. Return to the Jenkins master terminal.

19. Generate an Ed25519 key pair:

```
ssh-keygen -t ed25519 -f ~/.ssh/sshkey
```

20. When prompted for a passphrase, press Enter if you want no passphrase.

21. The command creates:

- ~/.ssh/sshkey — private key
- ~/.ssh/sshkey.pub — public key

9. Put the Public Key on the Agent

22. On the master, display the public key:

```
cat ~/.ssh/sshkey.pub
```

23. Copy the entire output.

24. On the agent, create the SSH directory if necessary:

```
mkdir -p ~/.ssh  
chmod 700 ~/.ssh
```

25. Open the authorized keys file:

```
nano ~/.ssh/authorized_keys
```

26. Paste the public key into authorized_keys.

27. Save with Ctrl+O, press Enter, then exit with Ctrl+X.

28. Set the correct permission:

```
chmod 600 ~/.ssh/authorized_keys
```

10. Test Master → Agent SSH

29. From the Jenkins master:

```
ssh -i ~/.ssh/sshkey ubuntu@AGENT_PRIVATE_IP
```

30. If the connection succeeds, exit the agent:

```
exit
```

11. Create the Jenkins Agent Node

31. In Jenkins, open:

Dashboard → Manage Jenkins → Nodes

32. Click New Node.

33. Enter the node name:

sagar

34. Select Permanent Agent.

35. Click Create.

12. Configure the Agent Node

- Name: sagar
- Description: Jenkins Docker build agent
- Number of executors: 1
- Remote root directory: /home/ubuntu/jenkins
- Labels: sagar
- Usage: Only build jobs with label expressions matching this node

The label must be "sagar" because the pipeline uses: agent { label "sagar" }

13. Configure SSH Launch

36. For Launch method, select:

Launch agents via SSH

37. For Host, enter the agent private IP if both EC2 instances are in the same VPC:

AGENT_PRIVATE_IP

38. Under Credentials, click Add → Jenkins.

14. Add the SSH Credential

- Kind: SSH Username with private key
- Username: ubuntu
- Private Key: Enter directly

39. On the Jenkins master, display the private key:

```
cat ~/.ssh/sshkey
```

40. Copy the entire private key, including the BEGIN and END lines.

41. Paste it into the Jenkins private-key field.

42. Add the credential and select it for the agent.

43. Click Save.

15. Confirm the Agent Connection

44. Open:

```
Manage Jenkins → Nodes → sagar
```

45. Confirm that the node shows Connected.

16. Install Docker on the Agent

46. SSH into the agent:

```
ssh -i your-key.pem ubuntu@AGENT_PUBLIC_IP
```

47. Install Docker using the official Docker instructions.

48. Verify Docker:

```
docker --version
```

49. Verify Docker Compose:

```
docker compose version
```

Docker and Docker Compose must be available on the agent because the pipeline runs docker build and docker compose up -d on the agent.

17. Allow Jenkins to Use Docker

50. On the agent, add the Jenkins user to the Docker group:

```
sudo usermod -aG docker jenkins
```

51. Restart/reconnect the Jenkins agent so the new group membership takes effect.

18. Create the Jenkins Pipeline Job

52. In Jenkins, open:

```
Dashboard → New Item
```

53. Enter a job name.

54. Select Pipeline.

55. Click OK.

56. Under Pipeline, select:

```
Definition → Pipeline script
```

19. Pipeline Script

57. Paste the following verified pipeline script into the Pipeline script field:

```
pipeline {
  agent { label "sagar" }

  stages {
    stage('code') {
      steps {
        echo "this is clonning the code:"

        git url: "https://github.com/sagarpyakurel/django-notes-app-for-jenkinsProject.git",
            branch: "main"

        echo "code Clonning successful"
      }
    }

    stage('Build') {
      steps {
        echo "this is building the code:"

        sh "docker build -t notes-app:latest ."
      }
    }

    stage('Test') {
      steps {
        echo " this is testing the code:"
      }
    }

    stage('Deploy') {
      steps {
        echo "this is deploying the code:"

        sh "docker compose up -d"
      }
    }
  }
}
```

The pipeline above is syntactically balanced: pipeline → stages → stage → steps are all properly closed.

20. Run the Pipeline

58. Click Save.

59. Click Build Now.

60. Jenkins will execute the pipeline on the agent labeled sagar:

```
Code
↓
Clone GitHub repository
↓
Docker Build
↓
Test
↓
Docker Compose Deploy
↓
Running application
```

21. Machine Responsibilities

Jenkins Master

- Jenkins web UI
- Pipeline orchestration
- Job management
- SSH connection to the agent

Jenkins Agent: sagar

- Git clone
- docker build
- docker compose up -d
- Application deployment

22. Final Replication Checklist

Master EC2

- Create EC2
- SSH into master
- Install Java JDK
- Install Jenkins
- Check Jenkins status
- Enable Jenkins

- Open port 8080
- Open Jenkins in browser
- Get initialAdminPassword
- Install suggested plugins
- Create admin account

Agent EC2

- Create second EC2
- SSH into agent
- Install Java JDK
- Install Docker
- Verify Docker Compose
- Allow Jenkins to use Docker

SSH Master → Agent

- `ssh-keygen -t ed25519 -f ~/.ssh/sshkey`
- Copy `sshkey.pub`
- Add the public key to `~/.ssh/authorized_keys` on the agent
- Set SSH permissions
- Test SSH from the master

Jenkins Node

- Manage Jenkins → Nodes
- New Node
- Name: `sagar`
- Permanent Agent
- Remote directory: `/home/ubuntu/jenkins`
- Label: `sagar`
- Launch agents via SSH
- Agent private IP
- SSH Username with private key
- Username: `ubuntu`
- Paste `~/.ssh/sshkey`
- Save
- Confirm Connected

Pipeline

- New Item
- Pipeline
- Definition: Pipeline script
- `agent { label "sagar" }`
- GitHub clone
- Docker build
- Test
- Docker Compose deploy
- Build Now

- Confirm SUCCESS

End of manual